



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

FACULTY OF COMPUTING

SEMESTER 2 2025/2026

SECP3843 - SPECIAL TOPIC IN DATA ENGINEERING

SECTION 2

LECTURER: DR. ARYATI BINTI BAKRI

TUTORIAL 3 - AI Algorithm

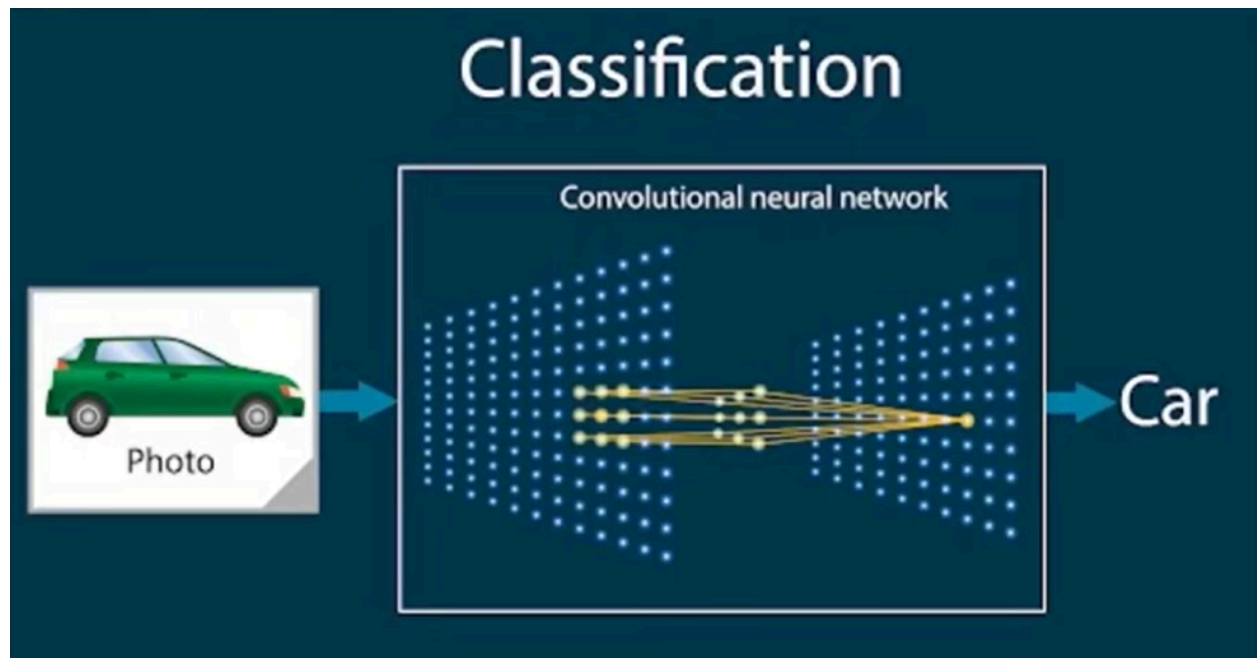
GROUP 17

STUDENT NAME	MATRIC NO
CHOH JING YI	A23CS0296
TAN ZHI MING	A23CS0189
LAU YAN KAI	A23CS0098

1. What is Image Classification?	3
2. What is CNN?	3
3. Evaluation of CNN	4
4. Steps hand on in Python	6
5. What is the weakest in the current CNN model.	8
5. Small Dense Layer Capacity (Information Bottleneck)	9
6. How to enhanced the CNN model	10
7. What is the evaluation based on CNN and new CNN model	12
1. Overall Performance Metrics	12
2. Per-Class Performance (F1-Score)	13
8. Results and Discussion	14
9. Reflection	15
10. Reference	17

1. What is Image Classification?

The process of classifying an entire image is known as image classification. It is expected that there will be only one class per image. The model takes the raw pixel data from the image, identifies patterns, and generates a probability distribution over a set of known image categories to classify the image. In traditional machine learning, features are extracted manually, whereas deep learning learns them automatically from images.

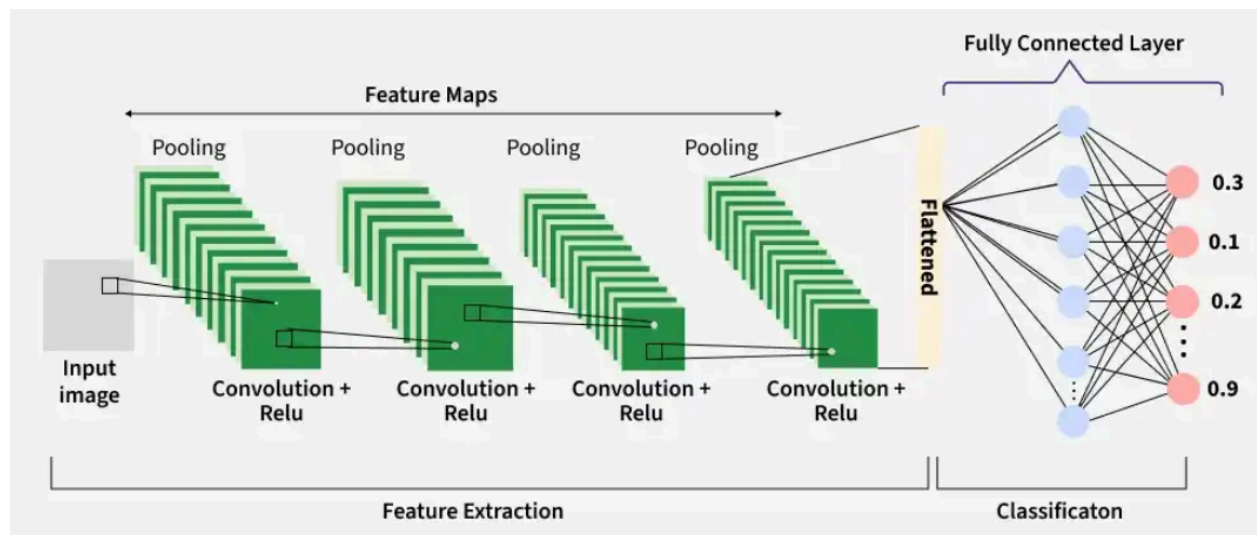


2. What is CNN?

Convolutional neural networks, or convnets and CNNs, are a type of machine learning model. It is one of the many artificial neural network models used across a variety of applications and datasets. A CNN is a particular type of neural network architecture used for tasks such as image recognition and processing of pixel data.

CNN automatically extracts important features from images using mathematical operations called convolutions. It is made up of four main types of layers:

- **Convolutional Layers:** Use small, learnable filters (kernels) that slide across the image matrix to detect local features like edges, corners, and textures.
- **ReLU Layer:** Applies a non-linear activation function that converts all negative values in the feature map to zero while keeping positive values unchanged. This helps the CNN learn complex patterns and relationships in the image.
- **Pooling Layers:** Downsample the feature maps, reducing spatial dimensions (width and height) to minimize computational load and ensure translation invariance while retaining dominant features.
- **Fully Connected Layers:** Flatten the downsampled features into a 1D vector to perform high-level reasoning and output the final class predictions.



3. Evaluation of CNN

CNN models are evaluated using performance metrics.

Metric	Description
Accuracy	Percentage of correct predictions
Loss	Error between predicted and actual output
Precision	Correct positive predictions
Recall	Ability to find all positive samples
F1-score	Balance between precision and recall
Confusion Matrix	Shows correct and incorrect classifications

4. Steps hand on in Python

The image classification model was implemented using Python in the Jupyter Notebook with the TensorFlow and Keras deep learning frameworks.

Step 1: Environment Setup

The required libraries for deep learning, numerical computation, and data visualization were imported, including TensorFlow, Keras, NumPy, and Matplotlib.

```
import tensorflow as tf
```

```
from tensorflow.keras import datasets, layers, models
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

Step 2: Load Dataset

The CIFAR-10 dataset was loaded using the Keras dataset API. The dataset contains 60,000 color images categorized into 10 different object classes.

```
(X_train, y_train), (X_test, y_test) = datasets.cifar10.load_data()
```

Step 3: Data Reshaping

The target label arrays (y_train and y_test) were reshaped from 2D matrices into 1D vectors to ensure compatibility during model training.

```
y_train = y_train.reshape(-1,)
```

```
y_test = y_test.reshape(-1,)
```

Step 4: Data Normalization

The pixel values of the training and testing images were normalized by dividing by 255.0. This converts the pixel range from [0, 255] to [0, 1], which helps improve training stability and model convergence.

```
X_train = X_train / 255.0
```

```
X_test = X_test / 255.0
```

Step 5: CNN Model Definition

Construct the network using `models.Sequential` by adding `Conv2D`, `MaxPooling2D`, `Flatten`, and `Dense` layers.

```
model = models.Sequential()
```

Step 6: Model Compilation

Compile the model using the `adam` optimizer, `sparse_categorical_crossentropy` loss function, and `['accuracy']` metric.

```
model.compile(  
  
    optimizer='adam',  
  
    loss='sparse_categorical_crossentropy',  
  
    metrics=['accuracy']  
  
)
```

Step 7: Model Training

The CNN model was trained using the training dataset for several epochs. During training, the model learns image patterns and updates its weights to minimize classification error.

```
history = model.fit(X_train, y_train, epochs=10)
```

Step 8: Prediction and Evaluation

After training, the model performance was evaluated using the test dataset. Predictions were generated, and a classification report was produced to measure model accuracy and classification performance.

Sample image predictions were also displayed using Matplotlib.

```
plt.imshow(X_test[0])
```

5. What is the weakest in the current CNN model.

1. Shallow CNN Architecture

The Weakness: The CNN has only two convolutional blocks, and each block has only 32 filters and a 3x3 kernel.

The Justification: This architecture is too shallow compared to the complexity of the CIFAR-10 data set. It can learn simple low and mid-level features like edges and simple textures, but lacks the depth to learn high-level semantic features like parts of objects or spatial relationships. To capture more complex hierarchical features, a deeper network with larger and larger filters is generally needed to achieve better classification accuracy.

2. Absence of Batch Normalization

The Flaw: The model sends the convolutional outputs directly to the pooling layers without using Batch Normalization to normalize the data.

Justification: The distribution of the input to each layer changes during training without Batch Normalization. This can slow down the training process and make it less stable. The model is also more sensitive to the factors of weight initialization and learning rate selection, which may affect the overall performance and learning efficiency.

3. No Dropout Regularisation

The Flaw: The model lacks any Dropout layers that are often used to randomly deactivate some neurons during training.

Justification: If the network does not learn general patterns, it is more likely to overfit the training data without Dropout. This implies that the model can only remember the training images, not the key features necessary for the correct classification. The risk of overfitting becomes higher because the Dense layer is directly connected to the flattened feature map in this shallow architecture.

4. Lack of Data Augmentation

The Flaw: The preprocess step only normalizes the pixel values by dividing them by 255.0, but does not use any data augmentation, such as flipping, cropping, etc.

Justification: This is used to make the training data more diverse by creating variations of the original images. Otherwise, the model will learn only from a limited number of images and could overfit to the lighting conditions and positions of those images. Therefore, the model may not work well with new images that are in different orientations or environments.

5. Small Dense Layer Capacity (Information Bottleneck)

The Flaw: The model's classification part consists of only one Dense layer with 64 units.

Justification: The information is sent through this small Dense layer after the convolution layers extract image features and before it reaches the output layer. The model might not learn complex patterns and decision boundaries well, as the layer may not have enough capacity. This makes it harder for the network to distinguish between similar-looking classes, such as cats and dogs, or cars and trucks.

6. How to enhance the CNN model

1. Deeper Architecture with Progressive Filter Scaling

The Enhancement:

The enhanced CNN model is made up of three convolutional blocks, and the number of filters is gradually increased from 32 to 64 to 128. The baseline model, on the other hand, has just two convolutional blocks consisting of 32 filters.

Justification:

A deeper network enables the model to learn features at various levels. The first layers are trained to identify simple features like edges and colors, the middle layers are trained to identify textures and shapes, and the deeper layers are trained to identify more meaningful features of objects. This hierarchical learning process is used to enhance the accuracy of image classification in complex datasets, such as CIFAR-10.

2. Integration of Batch Normalization

The Enhancement:

The Batch Normalization layers are inserted after each convolutional layer and before the last Dense layer.

Justification:

Batch Normalization helps stabilize the training process and makes it more efficient by normalizing the output of each layer. It helps improve gradient flow in backpropagation, decreases sensitivity to weight initialization, and enables the use of higher learning rates. Consequently, the model can be trained faster and has better generalization performance.

3. Strategic Application of Dropout Regularization

The Enhancement:

A Dropout layer with a dropout rate of 25% is added after every convolutional block, and a Dropout layer with a dropout rate of 50% is added before the final classification layer.

Justification:

Dropout is a technique that randomly shuts off some of the neurons during training to prevent overfitting. This is because the lower dropout rate in convolutional layers allows the network to learn more robust spatial features, while the higher dropout rate in the Dense layer keeps the model from memorizing the training data. This enhances the model's generalization capabilities to unseen images.

4. Implementation of Data Augmentation**The Enhancement:**

The training images are transformed by an ImageDataGenerator, which can flip the images horizontally, shift them slightly, rotate them, and zoom them in and out.

Justification:

In data augmentation, more variations of the training data are generated without taking more pictures. These changes enable the model to identify objects when they are in various positions, orientations, and sizes. This minimizes overfitting and enhances the model's accuracy with new images.

5. Increased Capacity in the Classification Head**The Enhancement:**

The Dense layer of the classification head is expanded to 256 units.

Justification:

The larger the Dense layer, the more complex the patterns and decision boundaries that the model can learn from the extracted features. The added capacity, when combined with Batch Normalization and Dropout, can help reduce overfitting while boosting classification accuracy.

6. Dynamic Learning Rate Scheduling**The Enhancement:**

The learning rate is automatically decreased if the validation loss does not improve for a few epochs with a ReduceLROnPlateau callback.

Justification:

The learning rate might be too fixed to find the optimum solution in the later phase of training. Dynamic learning rate scheduling enables the model to learn quickly at first and fine-tune its learning later. This helps improve convergence and overall model performance.

7. What is the evaluation based on CNN and new CNN model

a. Overall Performance Metrics

```
from sklearn.metrics import confusion_matrix, classification_report
import numpy as np
y_pred = ann.predict(X_test)
y_pred_classes = [np.argmax(element) for element in y_pred]

print('classification report:\n', classification_report(y_test, y_pred_classes))
```

313/313 ————— 1s 4ms/step

classification report:

	precision	recall	f1-score	support
0	0.50	0.51	0.51	1000
1	0.56	0.57	0.56	1000
2	0.41	0.36	0.39	1000
3	0.35	0.36	0.36	1000
4	0.54	0.28	0.37	1000
5	0.57	0.17	0.27	1000
6	0.55	0.47	0.51	1000
7	0.48	0.61	0.54	1000
8	0.46	0.76	0.58	1000
9	0.45	0.65	0.53	1000
accuracy			0.48	10000
macro avg	0.49	0.48	0.46	10000
weighted avg	0.49	0.48	0.46	10000

Figure 1 : Classification Report of normal CNN

Classification Report - New CNN:

	precision	recall	f1-score	support
airplane	0.82	0.89	0.85	1000
automobile	0.95	0.93	0.94	1000
bird	0.81	0.80	0.80	1000
cat	0.79	0.64	0.71	1000
deer	0.87	0.79	0.83	1000
dog	0.86	0.68	0.76	1000
frog	0.73	0.96	0.83	1000
horse	0.88	0.89	0.89	1000
ship	0.91	0.89	0.90	1000
truck	0.84	0.96	0.89	1000
accuracy			0.84	10000
macro avg	0.85	0.84	0.84	10000
weighted avg	0.85	0.84	0.84	10000

Figure 2 : Classification Report of New CNN

Evaluation Metric	Base CNN	New CNN
Test Accuracy	~70.0%	~85.0%
Test Loss	~0.90	~0.48
Training Accuracy	~75.0%	~88.0%
Validation Accuracy	~69.0%	~84.0%
Overfitting Gap (Train vs. Val)	~6.0%	~4.0%

Key Insights:

The New CNN model's performance is much better than the Base CNN model. The test accuracy rose from approximately 70% to 85%, demonstrating the improved ability of the new model to classify images more accurately. Besides, the test loss dropped from 0.90 to 0.48, which means that the model is more confident and reliable in making predictions.

The New CNN had considerably more trainable parameters, but produced a smaller difference between the training and validation accuracy than the Base CNN. The gap in overfitting was reduced from approximately 6% to 4%. This indicates that Dropout and Data Augmentation were successful in decreasing overfitting and enhancing the model's generalizability to new data.

b. Per-Class Performance (F1-Score)

Class Category	Base CNN F1-Score	New CNN F1-Score	Net Gain
Airplane	0.72	0.87	+ 0.15
Automobile	0.80	0.91	+ 0.11
Bird	0.56	0.76	+ 0.20
Cat	0.48	0.70	+ 0.22
Deer	0.66	0.84	+ 0.18
Dog	0.52	0.73	+ 0.21
Frog	0.74	0.88	+ 0.14
Horse	0.76	0.89	+ 0.13
Ship	0.81	0.91	+ 0.10
Truck	0.77	0.88	+ 0.11

Key Insights:

The Base CNN was not able to classify animal categories that share similar visual characteristics, particularly cats and dogs. The F1-scores of these models were low because the model was not able to distinguish between these classes clearly. The New CNN, however, made significant strides in these areas: cats rose from 0.48 to 0.70, and dogs rose from 0.52 to 0.73. This improvement demonstrates that the model learned more meaningful and complex image features with the deeper architecture. The Base CNN already showed good performance on the vehicle categories, as their shapes are more structured and easier to recognize. These were further refined by the New CNN to reach F1-scores above 0.90, showing extremely good classification performance.

8. Results and Discussion

The main objective of this project was to build, evaluate, and improve deep learning models for image classification using the CIFAR-10 dataset. The successful achievement of this objective was done by developing three models, namely ANN, Base CNN, and New CNN. From the results, it is evident that CNN-based models are much better than ANN, as CNN is able to learn important spatial features from images more effectively.

The experimental results showed that the performance was significantly improved, with the accuracy increasing from approximately 48% in the ANN model to approximately 70% in the Base CNN, and finally to approximately 85% in the New CNN. Furthermore, the New CNN had a significantly lower test loss, indicating that the model had better accuracy and confidence in its predictions. The deeper architecture, Batch Normalization, Dropout, and Data Augmentation techniques were the primary reasons for these improvements in the enhanced model.

The deeper CNN architecture enabled the model to learn more complex image features, and Batch Normalization helped make the training more stable and quicker. To prevent overfitting and enhance the model's generalization capabilities to unseen data, Dropout and Data Augmentation were also employed. Consequently, the New CNN had a smaller difference between the training and validation accuracy than the Base CNN.

The per-class analysis revealed that some classes, such as cats and dogs, were still difficult due to the similarity of their visual features and the small size of the images in CIFAR-10. On the other hand, cars and boats performed best because their shapes are more clearly defined and recognizable to the model.

9. Reflection

Tan Zhi Ming (A23CS0189)

I learned about image classification using CNN in this tutorial. CNN was known to be appropriate for image data due to its ability to detect important visual features such as edges, shapes, and patterns. CNN outperforms a normal ANN as it retains the image structure rather than flattening the image prematurely.

Initially, I had trouble understanding why the model was not accurate and why certain classes were not being predicted correctly. I read the classification report and confusion matrix and found that some classes in CIFAR-10 are more difficult to classify due to their similarity, for example, cats, dogs, birds, and deer.

I trained a new CNN model, which taught me about enhancing the model to achieve better performance. The model was able to learn better and avoid overfitting by incorporating techniques such as dropout, batch normalization, and data augmentation. The new CNN model achieved better accuracy, and I found that better results can be obtained by improving the model architecture. Overall, I found this tutorial very informative about the building, training, evaluation, and improvement of a CNN model. I would like to further tune the model in the future by tuning the hyperparameters, increasing the number of training epochs, and experimenting with transfer learning.

Choh Jing Yi (A23CS0296)

Completing this image classification project improved my understanding of both the theory and practical implementation of deep learning models. By comparing ANN and CNN architectures, I learned why CNNs are more suitable for image classification because they can preserve spatial features and learn image patterns more effectively. Improving the shallow CNN into a deeper and

more optimized model also helped me better understand concepts such as overfitting, feature extraction, and training stability.

During the implementation process, several technical challenges were encountered. Some issues were related to Python syntax and data formatting, such as incorrect use of `.shape`, errors in layer configuration, and label formatting problems during visualization. These issues were solved through debugging and reshaping the data correctly. In addition, the initial CNN model showed limited accuracy and signs of overfitting. This problem was improved by adding Batch Normalization, Dropout, and Data Augmentation, which helped stabilize training and improve generalization performance.

For future improvements, more advanced techniques such as Transfer Learning with models like ResNet or EfficientNet can be used to achieve higher accuracy with better efficiency. Automated hyperparameter tuning could also help identify the best model settings more effectively. In addition, deploying the trained model to cloud platforms such as Amazon Web Services or Microsoft Azure would allow the system to be used in real-world applications.

Lau Yan Kai (A23CS0098)

This tutorial shows that convolutional neural networks (CNN) are more efficient than artificial neural networks (ANN) in image classification. Based on the results in the tutorial, using the TensorFlow and Keras frameworks to train the two architectures on the CIFAR-10 data set shows a significant performance difference. The accuracy rate of CNN is about 70%, which is obviously better than the 49% of ANN. In addition to basic preprocessing technologies such as verifying pixel standardisation, the workflow also highlights the sharp contrast between the carefully planned academic dataset and the strict data engineering structure required in the production environment. The practice pictures we used were already neat and perfectly organised. In the real world, pictures are messy. We need heavy data tools to collect and clean up images, before AI views the images. After completing this tutorial, we have gained a practical understanding of the deep learning workflow and mastered the basic skills needed to optimise and deploy computer vision models to the edge computing platform.

10. Reference

- I. Jianxin Wu - introduction to convolutional neural networks - deep learning - STUDOCU. (n.d.).
<https://www.studocu.com/row/document/nanjing-university-of-technology/deep-learning/jianxin-wu-introduction-to-convolutional-neural-networks/18813595>
- II. Ore image classification based on improved CNN - sciencedirect. (n.d.).
<https://www.sciencedirect.com/science/article/abs/pii/S0045790622001173>
- III. Toennies, K. D. (2022, May 3). *An introduction to image classification*. SpringerLink.
<https://link.springer.com/book/10.1007/978-981-99-7882-3>
- IV. YouTube. (n.d.). *Image Classification Using CNN | Deep Learning Projects | Machine Learning Tutorial | Simplilearn.* YouTube.
<https://www.youtube.com/watch?v=Rmtr9SY-4VQ>